

Inigo Quilez :: articles :: 2D distance functions in L-infinity norm



Intro

This is a list of 2D SDF primitives in L^∞ -norm. While the L^∞ -norm produce very different distance fields to regular **Euclidean SDFs**, they are still fields and can be used to do raymarching, rasterization or collision detection. And just like Euclidean metric, the L^∞ -norm has its own set of advantages and disadvantages when dealing with SDFs.

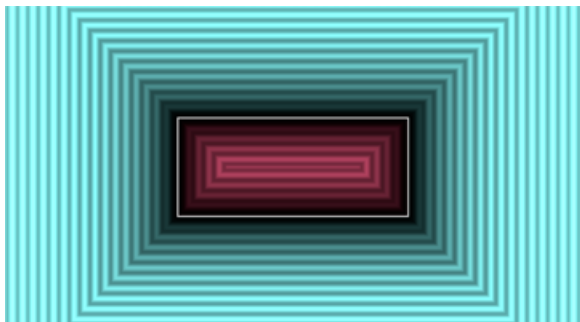
For example, L^∞ -norm work great on axis aligned algorithms, such as acceleration structure traversal (grids, or AABB trees or space partitioning). They are also able to represent many SDFs with simpler equations than the Euclidean metric, or even provide closed form analytic solutions where one doesn't exist in Euclidean metric. For example, Euclidean SDFs of ellipses and quadratic bezier curves need solving cubic equations, which can be slow and unstable, but in L^∞ -norm they are simple quadratic equations. In fact, in general, any shape that can be raycasted accepts an exact SDF in L^∞ -norm even when it might not have an analytic Euclidean SDF at all; and the computation usually boils down to a combination of bounding box checks and one or two simple raycasts in the $(\pm 1, \pm 1)$ directions.

On the other hand, the biggest drawback to L^∞ -norm SDFs is that rotations of primitives cannot be trivially computed, and require complicated implementations, as far as I can tell. One can often rotate the primitive and apply a distance correction factor of $\max(\text{abs}(\cos(\alpha) + \sin(\alpha)), \text{abs}(\cos(\alpha) - \sin(\alpha)))$, but the handling of the rotated primitive's bounding box in the general case seems complicated from what I've seen so far. Luckily, scaling and translating work just fine.

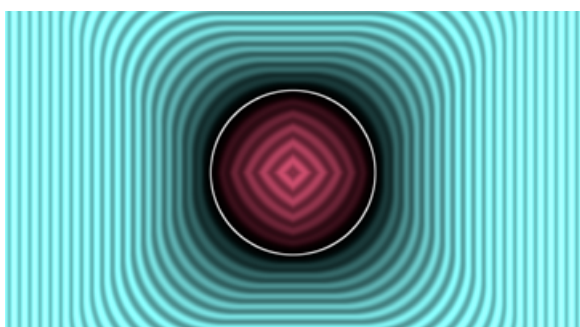
Anyway, here is the list of primitives I've derived so far. It did the work on a single night of obsessive play (including borrowing an idea from Shadertoy user "oneshade" for the $\max(p, (p-r).yx)$ trick). So the list is not exhaustive and some of the implementations can definitely be improved (the segment and arc SDFs in particular). When I do another such math-night session I'll hopefully improve them and update the reference code here. Regardless, you can find all of the primitives below in this Shadertoy playlist too:

<https://www.shadertoy.com/playlist/XXccDH>.

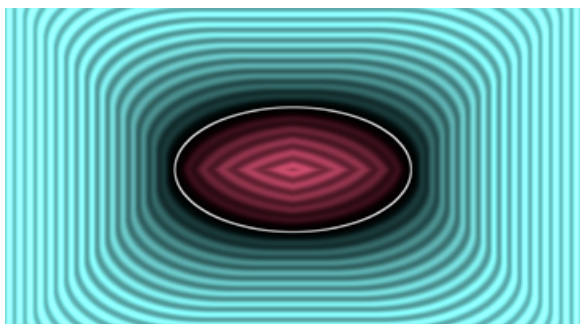
Primitives

**Box - exact** (<https://www.shadertoy.com/view/Nlj3WR>)

```
float sdBox( in vec2 p, in vec2 rad )
{
    p = abs(p)-rad;
    return max(p.x,p.y);
}
```

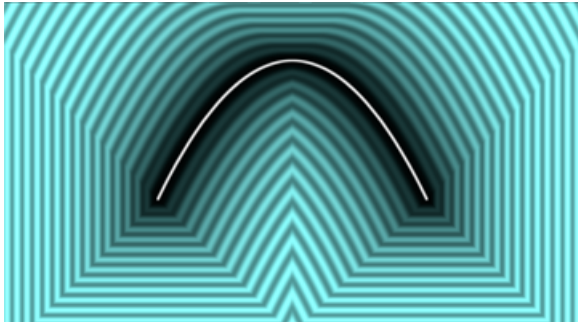
**Circle - exact** (<https://www.shadertoy.com/view/Nt2GWR>)

```
float sdCircle( in vec2 p, in float r )
{
    p = abs(p);
    p = max(p,p.yx-r);
    float a = 0.5*(p.x+p.y);
    float b = 0.5*(p.x-p.y);
    return a - sqrt(r*r*0.5-b*b);
}
```

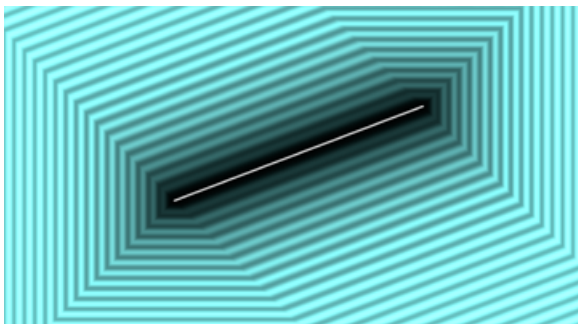


Ellipse - exact (<https://www.shadertoy.com/view/7tj3DR>)

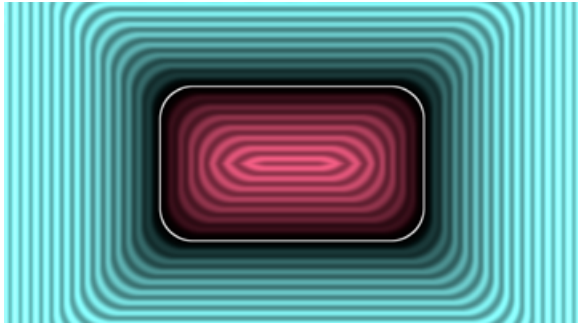
```
float sdEllipse( in vec2 p, in vec2 r )
{
    p = abs(p);
    p = max(p, (p-r).yx);
    float m = dot(r,r);
    float d = p.y-p.x;
    return p.x - (r.y*sqrt(m-d*d)-r.x*d)*r.x/m;
}
```

**Parabolic segment - exact** (<https://www.shadertoy.com/view/NsGyD3>)

```
float sdParabola( in vec2 p, float wi, float he )
{
    p.x = abs(p.x);
    float a = p.x-p.y-wi;
    float b = p.y-p.x-he;
    if( a>0.0 ) return max(abs(p.x-wi),abs(p.y));
    if( b>0.0 ) return max(abs(p.y-he), p.x );
    return p.x + (wi-sqrt(wi*wi-4.0*he*b)) * 0.5*wi/he;
}
```

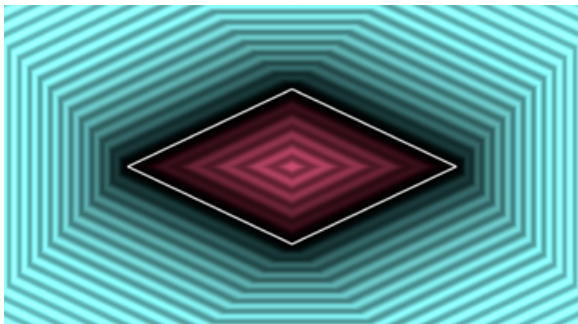
**Segment** (<https://www.shadertoy.com/view/7l2GWR>)

```
float sdLine( in vec2 p, vec2 a, vec2 b )
{
    vec2 pa = p-a, ba = b-a;
    float s = (ba.x*ba.y>0.0)?1.0:-1.0;
    float h = clamp( (pa.y+s*pa.x)/(ba.y+s*ba.x), 0.0, 1.0 );
    vec2 q = abs(pa-h*ba);
    return max(q.x,q.y);
}
```



Rounded Box - exact (<https://www.shadertoy.com/view/sl2GWz>)

```
float sdRoundBox( in vec2 p, in vec2 ra, in float rb )
{
    p = abs(p) - ra;
    p = max(p,p.yx-rb);
    float a = 0.5*(p.x+p.y);
    float b = 0.5*(p.x-p.y);
    return a - sqrt(rb*rb*0.5-b*b);
}
```



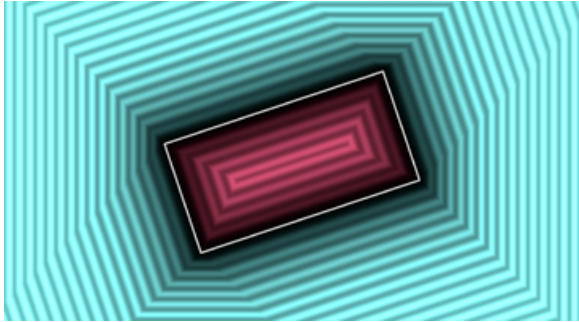
Rhombus - exact (<https://www.shadertoy.com/view/7tj3Wz>)

```
float sdRhombus( in vec2 p, float w, float h )
{
    p = abs(p); p.x -= w;
```

```

float f = clamp( (p.y-p.x)/(h+w), 0.0, 1.0 );
vec2 q = abs(p-f*vec2(-w,h));
return max(q.x,q.y)*((h*p.x+w*p.y>0.0)?1.0:-1.0);
}

```

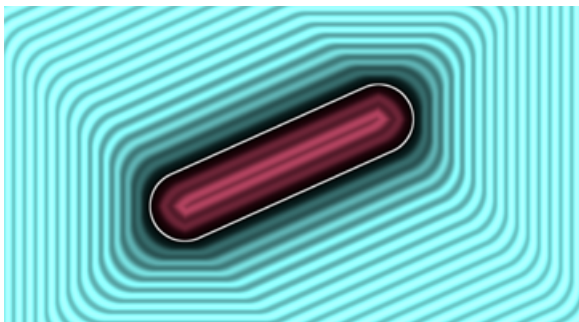


Oriented Box - exact (<https://www.shadertoy.com/view/NIBGDh>)

```

float sdOrientedBox( in vec2 p, in vec2 r, float ang )
{
    vec2 w = vec2(cos(ang),sin(ang));
    vec4 q = w.xyyx * p.xxyy;
    vec4 s = w.xyyx * r.xxyy;
    return max(
        // rotated rectangle
        max(abs(q.x+q.z)-r.x, abs(q.w-q.y)-r.y ) /
        max(abs(w.x-w.y), abs(w.x+w.y)),
        // axis aligned bbox
        max(abs(p.x)-max(abs(s.x-s.z),abs(s.x+s.z)),
            abs(p.y)-max(abs(s.y+s.w),abs(s.y-s.w)) ) );
}

```



Capsule (<https://www.shadertoy.com/view/ftS3DD>)

```

float sdCapsule( in vec2 p, vec2 va, vec2 vb, float rb )
{

```

```

// recenter primitive
p -= (vb+va)*0.5;
const float k = sqrt(0.5);
float l = length(vb-v-a);
vec2 u = (vb-v-a)/l;
vec2 v = vec2(-u.y,u.x);
l *= 0.5;
vec2 a = u*l;
vec2 w = v*rb;

// distance to body sides
float ss = (u.x*u.y>0.0)?1.0:-1.0;
float de = 1.0/(a.y+ss*a.x);
vec2 w1 = p-a-w;
vec2 w2 = p-a+w;
vec2 q1 = abs(w1+a*clamp(-(w1.y+ss*w1.x)*de, 0.0, 2.0));
vec2 q2 = abs(w2+a*clamp(-(w2.y+ss*w2.x)*de, 0.0, 2.0));
float d1 = max(q1.x,q1.y);
float d2 = max(q2.x,q2.y);
float d = min(d1,d2);

// inside circular caps
vec2 pa = p - a;
vec2 pb = p + a;
float da = dot(pa,pa);
float db = dot(pb,pb);
if( min(da,db)<rb*rb )
{
    float s = 1.0; if( db<da ) {pa=pb;s=-1.0;}
    float b1 = 0.5*(pa.x+pa.y);
    float b2 = 0.5*(pa.x-pa.y);
    float c = dot(pa,pa) - rb*rb;
    vec2 t1 = vec2(-b1,b1)+sqrt(b1*b1-c*0.5);
    vec2 t2 = vec2(-b2,b2)+sqrt(b2*b2-c*0.5);
    // 4 solution. Up to 3 can be valid
    if( s*dot(pa+vec2( 1, 1)*t1.x,-u)<0.0 ) d=min(d,t1.x);
    if( s*dot(pa+vec2(-1,-1)*t1.y,-u)<0.0 ) d=min(d,t1.y);
    if( s*dot(pa+vec2( 1,-1)*t2.x,-u)<0.0 ) d=min(d,t2.x);
    if( s*dot(pa+vec2(-1, 1)*t2.y,-u)<0.0 ) d=min(d,t2.y);
    d = -d;
}
// outside circular caps
else
{
    vec2 qa = abs(mat2(u.x,-u.y,u.y,u.x)*p) - vec2(1,rb);
    float di = max(qa.x, qa.y);
    // outside circular caps AND outside of body
    if( di>0.0 )
    {
        float dc;

```

```

// first cap
pa = abs(pa);
if( abs(pa.y-pa.x)<rb )
{
    float b = 0.5*(pa.x+pa.y);
    float c = dot(pa,pa) - rb*rb;
    dc = b - sqrt(b*b-c*0.5);
}
else
{
    dc = max(pa.x,pa.y)-rb;
}
d = min(d,dc);

// sadly, we still need to test the second cap
pb = abs(pb);
if( abs(pb.y-pb.x)<rb )
{
    float b = 0.5*(pb.x+pb.y);
    float c = dot(pb,pb) - rb*rb;
    dc = b - sqrt(b*b-c*0.5);
}
else
{
    dc = max(pb.x,pb.y)-rb;
}
d = min(d,dc);
}
d *= sign(di);
}

return d;
}

```



Circle Arc - exact (<https://www.shadertoy.com/view/7tSGDD>)

```

float dist( in vec2 p, in vec2 c )
{
    p = abs(p-c);
    return max(p.x,p.y);
}

float sdArc( in vec2 p, float rb, float w, float an )
{
    vec2 u = vec2(cos(an),sin(an));
    vec2 v = vec2(-u.y,u.x);
    float h = sqrt(rb*rb-w*w);

    // bounding points: arc extremes
    float d = min(dist(p, u*w+h*v),
                  dist(p, u*w-h*v));
    // bounding points: cardinals
    if( -rb*u.x<w ) d = min(d,dist(p,vec2(-rb,0.0)));
    if(  rb*u.x<w ) d = min(d,dist(p,vec2( rb,0.0)));
    if( -rb*u.y<w ) d = min(d,dist(p,vec2(0.0,-rb)));
    if(  rb*u.y<w ) d = min(d,dist(p,vec2(0.0, rb)));

    // circular section
    float b1 = 0.5*(p.x+p.y);
    float b2 = 0.5*(p.x-p.y);
    float c = dot(p,p) - rb*rb;
    float h1 = b1*b1-c*0.5;
    float h2 = b2*b2-c*0.5;
    if( h1>0.0 )
    {
        vec2 t = vec2(-b1,b1)+sqrt(h1);
        if( dot(p+vec2( 1, 1)*t.x,u)<w ) d=min(d,abs(t.x));
        if( dot(p+vec2(-1,-1)*t.y,u)<w ) d=min(d,abs(t.y));
    }
    if( h2>0.0 )
    {
        vec2 t = vec2(-b2,b2)+sqrt(h2);
        if( dot(p+vec2( 1,-1)*t.x,u)<w ) d=min(d,abs(t.x));
        if( dot(p+vec2(-1, 1)*t.y,u)<w ) d=min(d,abs(t.y));
    }

    return d;
}

```